# Cybersecurity Log Project, Explained Simply

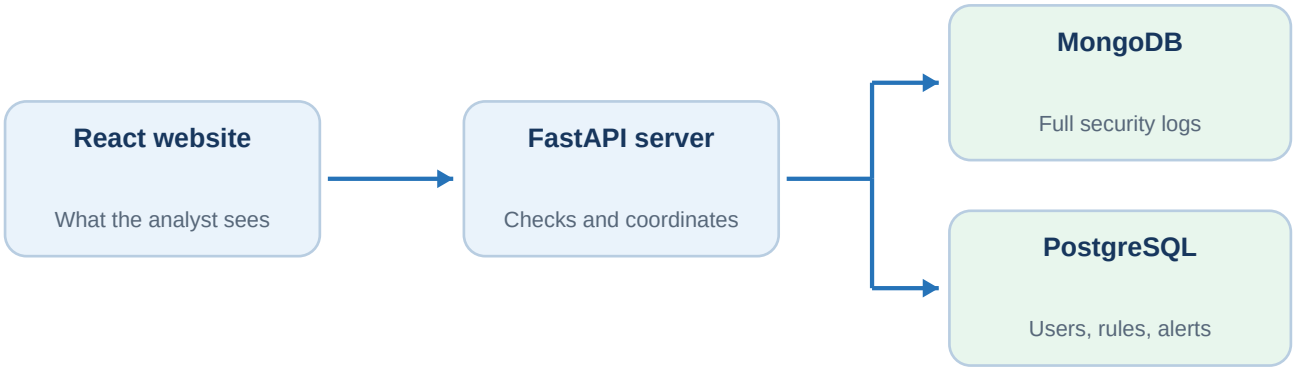
A clear guide for the three team members and the project review | 24 October 2026

**One-sentence idea:** The system collects security events, stores them, helps an analyst search quickly, spots suspicious patterns, and tracks what the team did about them.

## Why this project exists

A server, login system, or firewall may produce thousands of event messages every day. Reading them one by one is too slow. Our project puts them in one place so an analyst can find a specific event, see unusual behavior, and start an investigation.

## The four pieces




| | |
|---|---|
| **React website** | The screens for login, uploading logs, searching, alerts, incidents, and charts. |
| **FastAPI server** | The middle layer. It validates requests, applies detection rules, and talks to both databases. |
| **MongoDB** | Holds the complete security log messages. Different log types can have different fields. |
| **PostgreSQL** | Holds structured records: users, devices, rules, alerts, incidents, actions, and relationships. |

**Important distinction:** The working website uses MongoDB + PostgreSQL. A separate Oracle 11g script demonstrates the SQL/PLSQL topics required for Review II; Oracle is not the website's live database.

# What happens to one suspicious event?

Example: three failed logins from the same IP address within ten minutes.

### 1. A log arrives

A log is a record of something that happened. Example: **10:02, login failed, source IP 198.51.100.10**. The analyst can upload a CSV, JSON, or JSONL file, or send an event to the API.

### 2. The system checks and stores it

The server checks required fields, IP address, time, severity, and port. It rejects bad rows and tells the user how many were rejected. Valid logs are converted to a consistent format and saved in MongoDB.

### 3. An index helps us find it

An **index** is like the index at the back of a textbook: it points to relevant records without reading every record. The system indexes common search fields such as time, source IP, event type, and severity. Search results arrive in small pages.

### 4. A rule detects the pattern

The detection engine counts failed logins from that IP in the chosen time window. When the count reaches the configured threshold, it creates an alert. Other rules look for connections to many ports or unusually high activity.

### 5. An analyst investigates

The alert shows which rule fired and links to supporting logs. The analyst can acknowledge it, turn it into an incident, assign someone, and record notes or actions. Rerunning detection should not create a duplicate active alert for the same pattern.

### 6. The dashboard summarizes the situation

Charts show how many logs and alerts exist, severity levels, activity over time, and top source IPs. The dashboard helps prioritize work; it does not prove that an attack happened.

**Easy viva line:** A log is raw evidence. A rule finds a suspicious pattern. An alert asks an analyst to look. An incident records the investigation and response.

# The database concepts in simple words

These are the Review II terms your professor may ask you to explain.

| | |
|---|---|
| **DDL** | Commands that create the database structure: tables, columns, primary keys, foreign keys, checks, and indexes. |
| **Constraints** | Rules that protect data. A primary key identifies one row; a foreign key links related rows; UNIQUE prevents duplicates. |
| **DML** | Commands that change rows: INSERT adds data, UPDATE edits it, and DELETE removes it. |
| **JOIN** | Combines related rows, such as an alert with its detection rule and incident. |
| **Aggregation** | Summarizes rows, such as COUNT of failed logins grouped by source IP. |
| **Subquery** | A query inside another query, used here to compare a device's log count with an average. |
| **Function** | A named calculation that returns a value, such as a failed-login count or incident risk score. |
| **Procedure** | A named operation that performs work, such as scanning logs or refreshing incident priorities. |
| **Cursor** | A way for a procedure to process query results one row at a time. |
| **Trigger** | Code the database runs automatically after or before a change, such as recording alert status history. |

**How we show this:** PostgreSQL has the database operations used by the app and a Review II demo. The separate Oracle 11g script creates sample tables and data, runs queries, and demonstrates a function, procedures, a cursor, and triggers. Both paths passed live automated database checks.

# Who built what - and what to show

The three member branches preserve individual work; **main** contains the integrated project.

| | |
|---|---|
| **Member 1: Midhun** | Log input, validation, MongoDB storage, indexes, search, device/source records, synthetic data, and the one-million-log benchmark. |
| **Member 2: Shraavanth** | Login and roles, detection rules, suspicious-pattern checks, alert creation, evidence, and duplicate prevention. |
| **Member 3: Lakshman** | Incident and action records, analytics API, and the React screens and dashboard. |
| **Integrated main** | The combined application, Review II SQL work, tests, reports, and final fixes. |

## A simple review demonstration

**1.** Show the website and explain the two databases.
**2.** Log in, upload a small synthetic log file, and search by IP or time.
**3.** Run detection, open an alert, and show its supporting logs.
**4.** Create an incident, add an action, and show the dashboard.
**5.** Explain the SQL function, procedure, cursor, and trigger; show the passing Oracle 11g run.

## What was verified

The backend suite has 79 passing tests. GitHub checks passed with real MongoDB 7 and PostgreSQL 16, a Chromium browser walkthrough, and a one-million-log indexed search benchmark. The Oracle XE 11g Review II script also passed in a clean schema, including its DDL, DML, queries, and PLSQL checks.

For one benchmark query, the median indexed search took **0.396 ms**, versus **188.423 ms** for a forced full scan on the same CI runner. This comparison is for that test query, not a promise about every search.

**Project status:** The agreed academic prototype and both Review II database demonstrations are implemented and tested. A trained machine-learning model and enterprise deployment were outside the stated scope. The remaining step is your faculty presentation and evaluation.

Full technical report: github.com/midhungereketi/Intelligent-Cybersecurity-Log-Management-Systems/blob/main/docs/DETAILED_PROJECT_REPORT.md

Oracle 11g execution: github.com/midhungereketi/Intelligent-Cybersecurity-Log-Management-Systems/actions/runs/40022509813